

Agent IA: Déploiement plateforme, architecture, configuration, utilisation

Module 2

Fonctionnement interne des grands modèles de langage (LLMs)



24 Août – 1er Septembre 2026

Cédric Manouan

Contenu

- 1) Les architectures Transformer ;
 - a) Tokenisation et enchâssement (embeddings) ;
- 2) Méthodes d'apprentissage
 - a) Fine-tuning et adaptation des modèles;
- 3) Gestion du contexte et mémoire conversationnelle ;
- 4) Évaluation des performances des modèles.
- 5) Modèles Propriétaires (fermés)/Modèles ouverts

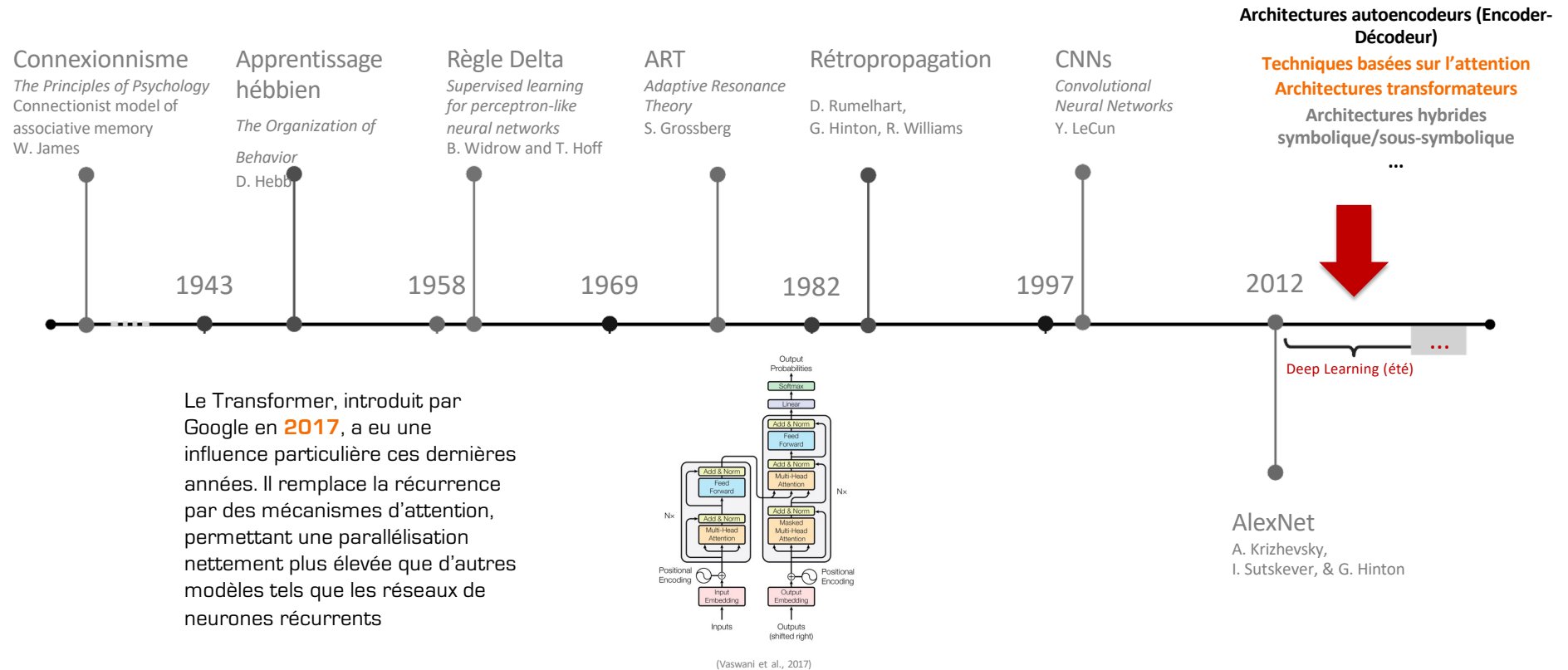
Rappel

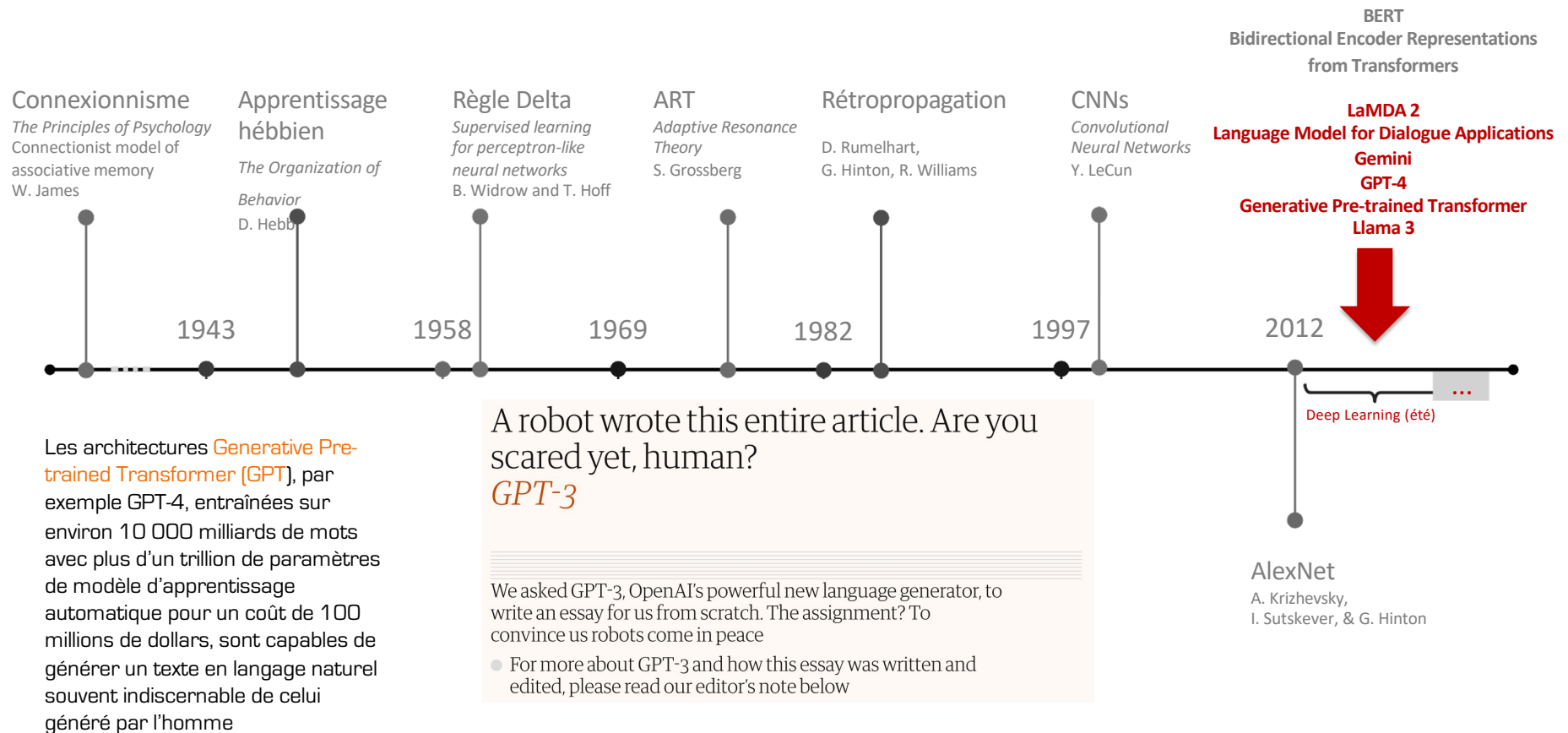
- ❑ Des modèles plus larges avec la capacité de **capturer des relations complexes et subtiles** au sein des données
- ❑ Ces modèles ont tendance à **mieux généraliser** au-delà des exemples vus pendant l'entraînement
- ❑ Ils sont volumineux peuvent être **pré-entraînés sur d'énormes ensembles de données** et ensuite ajustés pour des tâches spécifiques (fine-tuning)
- ❑ Majoritairement rendu possible grâce aux progrès dans les techniques de **calculs distribués**

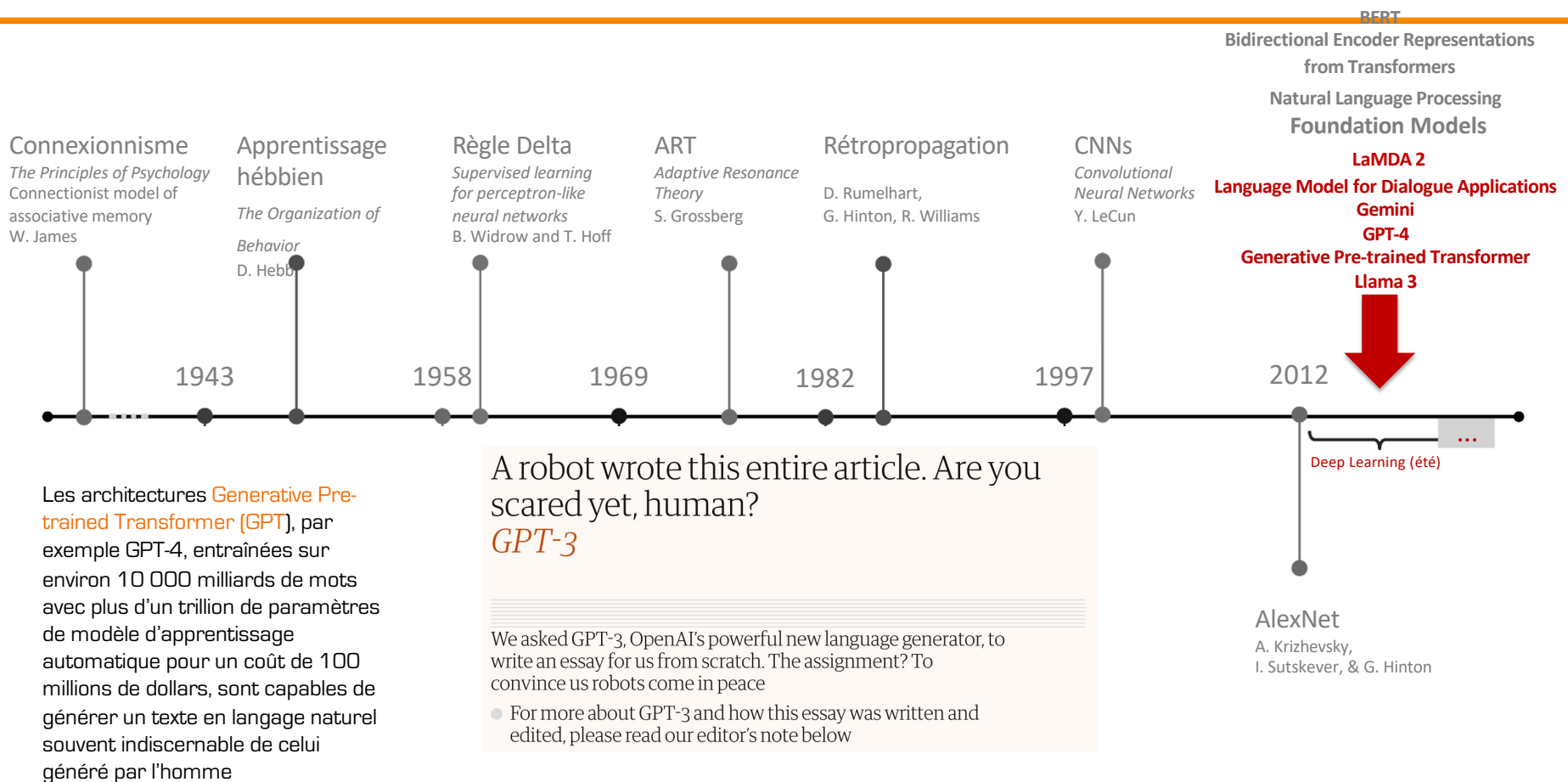
J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," arXiv preprint arXiv:2001.08361, 2020. <https://arxiv.org/abs/2001.08361>

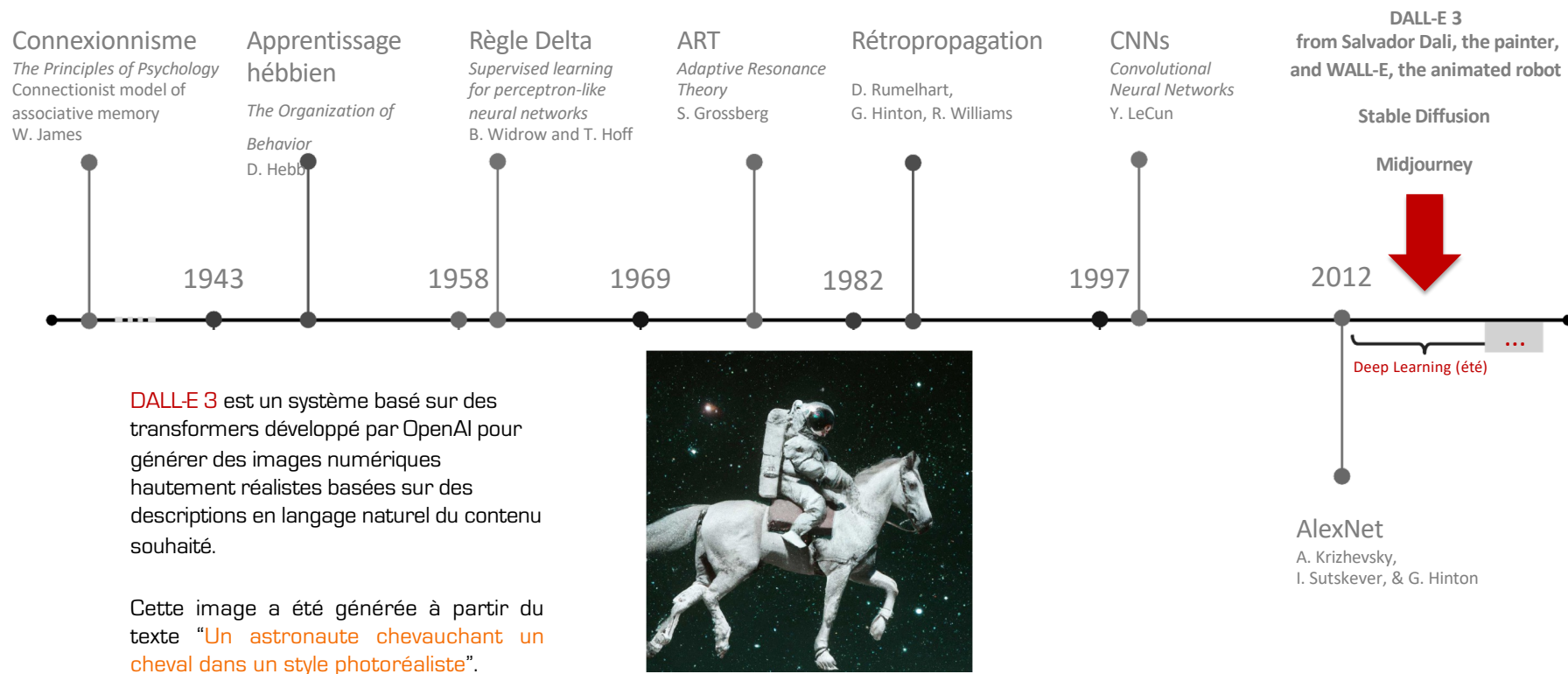
Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Rafuls, A., Kessler, S., ... & Sparks, E. (2022). Training Compute-Optimal Large Language Models. arXiv preprint arXiv:2203.15556.

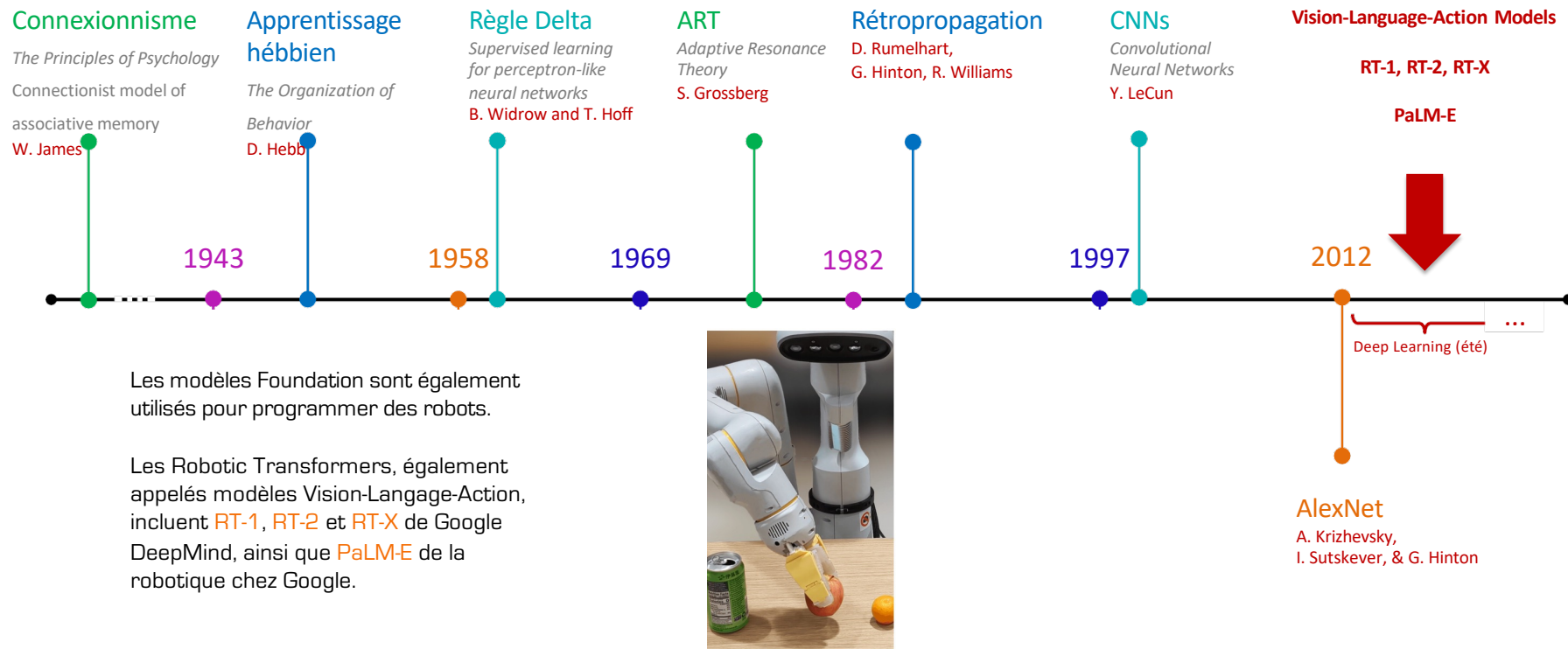
W. Chen, J. Tian, Y. Peng, B. Yan, C.-H. H. Yang, and S. Watanabe, "OWLS: Scaling Laws for Multilingual Speech Recognition and Translation Models," 2025. arxiv.org/abs/2502.10373





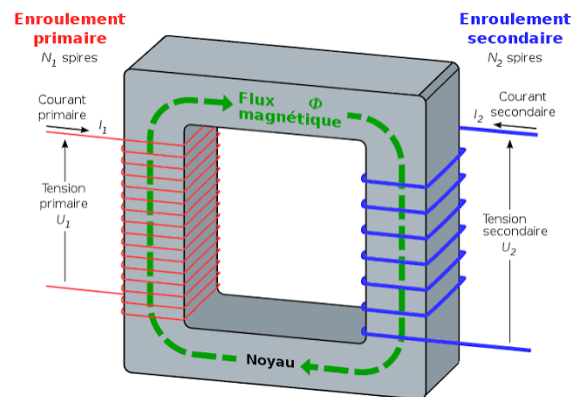




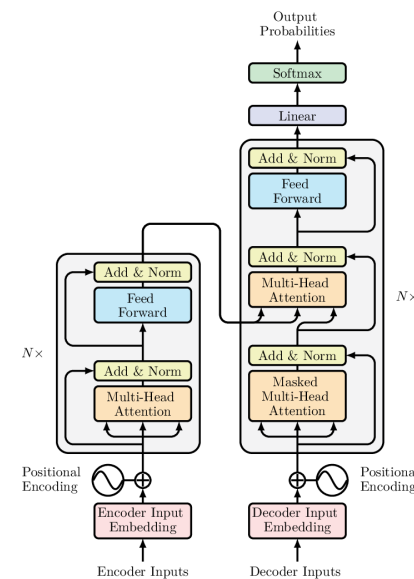


Transformateur électrique vs Transformer

L'analogie entre les deux réside dans leur fonction fondamentale : **adapter, convertir et acheminer un flux d'informations ou d'énergie** d'une forme à une autre avec un haut rendement.



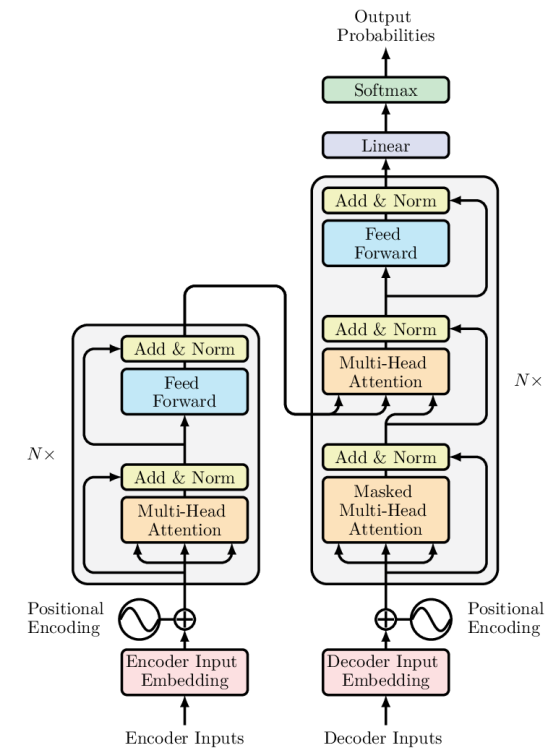
Transformateur électrique



Transformer

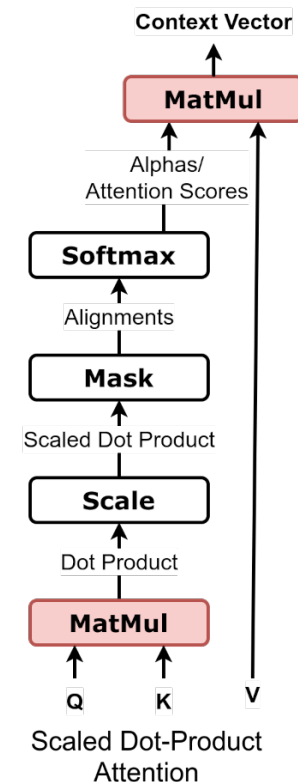
Le Transformer

- ❑ **Type:** Encoder-Decoder
- ❑ **Rôle :** Transforme une séquence de données d'entrée (par exemple, une phrase en français) en une séquence de sortie (la même phrase traduite en anglais).
- ❑ **Principe :** Utilise le mécanisme d'**auto-attention** (self-attention) pour **lier dynamiquement chaque mot d'une phrase à tous les autres mots simultanément**, créant un réseau de relations sémantiques.



Le Transformer

- ❑ le mécanisme d'**auto-attention** permet au modèle de **comprendre le contexte** d'un mot en calculant sa relation avec tous les autres mots d'une phrase
- ❑ Pour chaque mot (représenté par un vecteur numérique), le système génère trois sous-vecteurs essentiels:
- ❑ **La Requête/Query (Q)** : Ce que le mot recherche.
- ❑ **La Clé/Key (K)** : Ce que le mot propose pour caractériser son identité.
- ❑ **La Valeur/Value - (V)** : Le contenu sémantique réel qu'il transmet si l'affinité est forte.

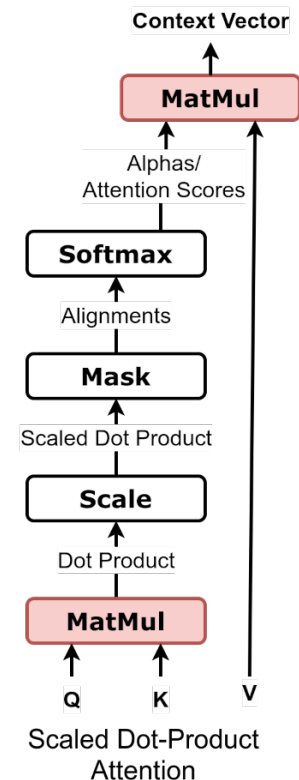


Le Transformer

- ❑ Le modèle mesure la pertinence entre la requête d'un mot et les clés de tous les autres en effectuant un produit scalaire.
- ❑ L'équation officielle de l'attention (issue du célèbre papier [Attention Is All You Need](#)) s'écrit:

$$Attention(Q, K, V) = softmax((QK^T)/sqrt(d_k))V$$

- QK^T : Calcule les scores de similarité (qui doit faire attention à qui).
- $sqrt\{d_k\}$: Facteur d'échelle pour stabiliser les gradients lors de l'apprentissage.
- **Softmax** : Convertit les scores en probabilités (poids d'attention) entre 0 et 1.
- xV : Multiplie ces poids par les valeurs pour obtenir une nouvelle représentation enrichie par le contexte.

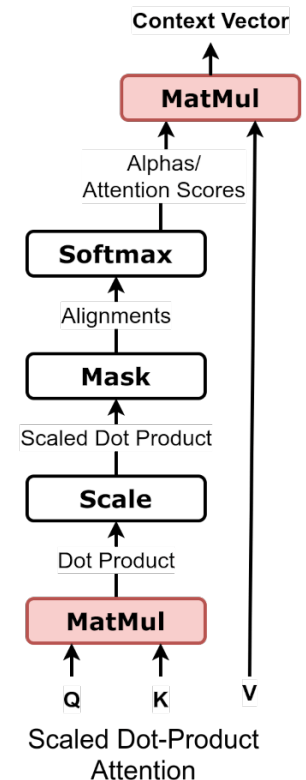


Le Transformer

Exemple concret : Dans la phrase "L'avocat l'a défendu au tribunal".

- ❑ le mot "avocat" va fortement s'aligner (haute attention) avec le mot "tribunal".
- ❑ Le modèle comprendra immédiatement qu'il s'agit du juriste et non du fruit.
- ❑ Ainsi, Un mot brut (vecteur d'embedding) se retrouve enrichi de son contexte sémantique

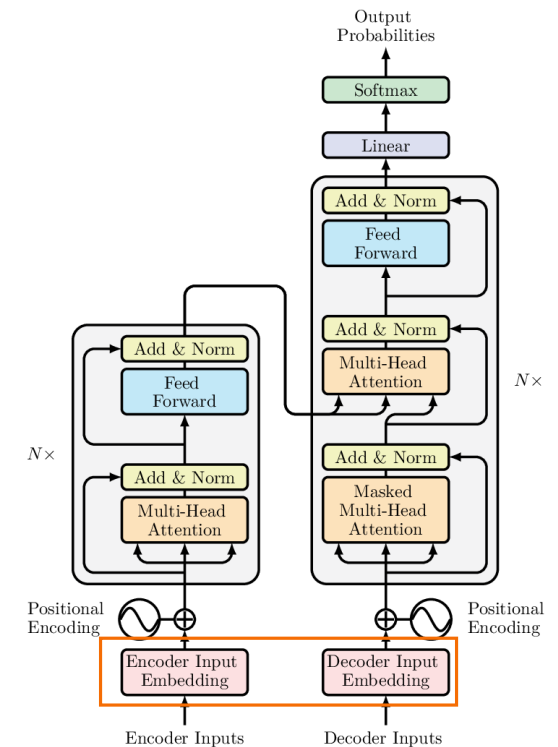
Mais qu'est-ce que le vecteur d'enchassement ou embedding?



La Tokenization

Avant d'appliquer le mécanisme d'attention, **le texte brut doit être traduit en nombres** interprétables par le réseau de neurones.

- ❑ La **tokenisation** découpe le texte en unités minimales appelées **tokens** (mots entiers, syllabes ou caractères).
- ❑ **Vocabulaire fixe** : Le modèle possède un dictionnaire strict (souvent entre 32 000 et 100 000 tokens ou plus).
- ❑ Chaque token est associé à un identifiant unique (un entier).
- ❑ On découpe les mots rares ou complexes en sous-mots (ex: "**transformation**" devient ["**transform**", "**ation**"]).



L'Enchâssement (Embedding)

Les identifiants numériques sont convertis en vecteurs continus à haute dimension (ex: 4096 dimensions).

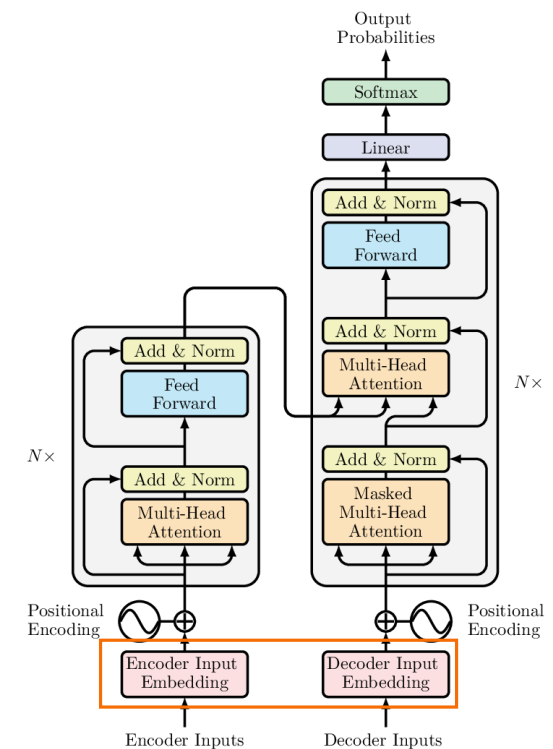
- ❑ **Espace sémantique** : Les tokens ayant un sens proche sont projetés à proximité géométrique dans cet espace.

Vocabulaire fixe : Le modèle possède un dictionnaire strict (souvent entre 32 000 et 100 000 tokens ou plus).

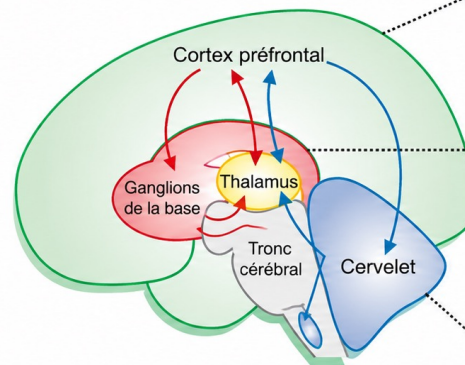
- ❑ **Codage positionnel (Positional Encoding)** : Comme le Transformer traite tous les tokens en parallèle, on ajoute un signal mathématique (sinusoïdal ou appris) au vecteur pour indiquer la position du mot dans la phrase.

Demo:

- ❑ <https://poloclub.github.io/transformer-explainer/>
- ❑ <https://projector.tensorflow.org/> (embeddings)



Méthodes d'apprentissage

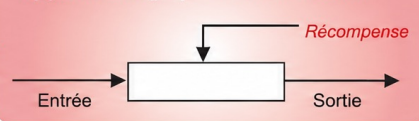


Apprentissage non supervisé



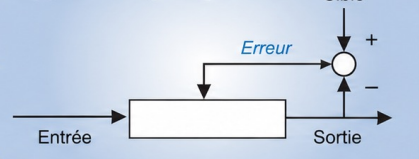
Regroupement à partir de motifs communs; Découvrir des **structures cachées** et des similarités par soi-même

Apprentissage par renforcement



Interaction avec un **environnement**; Essais et erreurs guidés par des **récompenses** ou des punitions

Apprentissage supervisé



Apprendre à partir d'**exemples étiquetés**; Prédire des résultats ou des catégories sur de nouvelles données

Fine-Tuning ou Adaptation des Modèles (1/5)

L'entraînement complet d'un LLM se déroule toujours en **deux grandes phases successives** : le **Pré-entraînement** (qui crée la structure et la culture générale) et **l'Alignement/Fine-Tuning** (qui spécialise le modèle et le rend sûr).

Phase 1 : Le Pré-entraînement (Pre-training)

- ❑ Cette phase est la plus **lourde**, la plus **coûteuse** (des millions de dollars en serveurs) et **dure plusieurs mois**. Elle consiste à fabriquer un modèle de base (Base Model).
- ❑ **La collecte des données** : Aspiration de gigantesques volumes de données (Wikipedia, livres, articles de presse, code informatique GitHub, Reddit). Ce corpus représente des milliers de milliards de **tokens**.
- ❑ **Objectif mathématique (Causal Language Modeling)** : Le modèle apprend à faire une seule tâche simple en boucle : **prédire le mot suivant** à partir des mots précédents.

Fine-Tuning ou Adaptation des Modèles (2/5)

Phase 2 : Le Fine-Tuning et l'Alignement

Cette phase transforme le modèle de base en un **Assistant Conversationnel** (comme ChatGPT, Claude ou Llama). Elle utilise principalement deux méthodes de tuning :

1. Le Fine-Tuning Supervisé (SFT - Supervised Fine-Tuning): On présente au modèle des milliers d'exemples de haute qualité rédigés par des humains sous forme de couples **[Instruction / Réponse attendue]**.

❑ Exemple d'instruction : "Résume ce texte en 3 phrases."

❑ Exemple de réponse : "Voici le résumé : 1... 2... 3..."

❑ **Résultat :** Le modèle modifie ses poids internes pour copier la structure de réponse d'un assistant humain idéal.

Fine-Tuning ou Adaptation des Modèles (3/5)

2. L'Alignement par Préférences Humaines

Pour s'assurer que le modèle est sûr, refuse de donner des instructions dangereuses (comme fabriquer une arme) et choisit le meilleur ton, on utilise l'une de ces deux techniques modernes.

RLHF (Reinforcement Learning from Human Feedback) :

- ☐ Le modèle génère plusieurs réponses possibles à une question.
- ☐ Des évaluateurs humains (ou un modèle d'évaluation) classent les réponses de la meilleure à la moins bonne.
- ☐ On entraîne un **Modèle de Récompense** (Reward Model) à comprendre ce que les humains préfèrent.
- ☐ On utilise l'**apprentissage par renforcement** pour ajuster le LLM afin qu'il maximise son score de récompense.

Fine-Tuning ou Adaptation des Modèles (4/5)

2. L'Alignement par Préférences Humaines

Pour s'assurer que le modèle est sûr, refuse de donner des instructions dangereuses (comme fabriquer une arme) et choisit le meilleur ton, on utilise l'une de ces deux techniques modernes .

DPO (Direct Preference Optimization) :

- ☐ Une méthode plus récente et plus élégante qui saute l'étape du modèle de récompense.
- ☐ Elle optimise **directement** le LLM en lui présentant des paires de données contenant une réponse "gagnante" (chosen) et une réponse "perdante" (rejected). Le modèle ajuste ses calculs mathématiques pour augmenter la probabilité de générer la bonne réponse et réduire celle de la mauvaise.

Fine-Tuning ou Adaptation des Modèles (5/5)

Les techniques d'optimisation matérielle (PEFT / LoRA)

- ☐ Puisqu'un LLM possède des dizaines de milliards de paramètres, modifier tous les poids lors du fine-tuning demande une mémoire vive (VRAM) gigantesque sur les cartes graphiques (GPU).
- ☐ Pour résoudre ce problème, on utilise le **PEFT (Parameter-Efficient Fine-Tuning)**, dont la méthode la plus célèbre est **LoRA (Low-Rank Adaptation)**.
- ☐ Au lieu de modifier la matrice géante de poids d'origine du modèle, on la fige (on la bloque en lecture seule).
- ☐ Puis on injecte à côté deux matrices de calcul minuscules et très fines.
- ☐ Lors du fine-tuning, **seules ces petites matrices LoRA apprennent**.
- ☐ Cela permet de diviser significativement les besoins en mémoire et de créer des "fichiers adaptateurs" très légers (quelques mégaoctets au lieu de plusieurs gigaoctets).

Gestion du contexte et de la mémoire conversationnelle (1/4)

La gestion du contexte et de la mémoire conversationnelle dans les grands modèles de langage (LLMs) repose sur une réalité technique simple : **les LLMs n'ont pas de mémoire à long terme innée** et ne "retiennent" rien d'une session à l'autre.

1. La fenêtre de contexte (Context Window)

- ☐ La fenêtre de contexte est la quantité maximale de données (mots ou morceaux de mots appelés **tokens**) qu'un modèle peut traiter en une seule fois.
- ☐ **La limite physique** : Si un modèle a une fenêtre de 8 000 tokens, tout ce qui dépasse cette taille est tout simplement ignoré ou coupe la conversation.
- ☐ **L'illusion de la mémoire** : Pour que le LLM se rappelle ce que vous avez dit dix messages plus tôt, l'application informatique (l'interface) prend discrètement **tout l'historique** de la discussion et le renvoie au modèle à chaque nouveau message.

Gestion du context et de la mémoire conversationnelle (2/4)

2. Les stratégies de mémoire conversationnelle

Renvoyer l'intégralité de l'historique devient vite coûteux, lourd et impossible si la discussion dure des jours.

❑ Mémoire tampon complète (Conversation Buffer Memory) :

- Principe : L'intégralité des messages passés est renvoyée à chaque fois.
- Limite : Saturation rapide de la fenêtre de contexte et augmentation des coûts.

❑ Fenêtre glissante (Conversation Window Buffer) :

- Principe : Le système ne conserve que les K derniers messages (par exemple, les 5 dernières questions/ réponses) et jette le reste.
- Limite : Le modèle oublie complètement le début de la conversation.

❑ Résumé de conversation (Conversation Summary Memory) :

- Principe : Un LLM secondaire (ou le même modèle en tâche de fond) rédige un résumé condensé de la discussion passée. Ce résumé est injecté au début de la nouvelle requête.
- Avantage : Gain de place immense tout en gardant le fil conducteur.

Gestion du context et de la mémoire conversationnelle (3/4)

3. Le KV Caching (Optimisation technique interne)

- ☐ À l'intérieur de l'architecture Transformer, recalculer les matrices d'attention (les vecteurs Clés/Keys et Valeurs/Values vus précédemment) pour tout l'historique à chaque nouveau mot serait extrêmement lent.
- ☐ Le **KV Caching** consiste à stocker en mémoire RAM (sur les cartes graphiques/GPU) les calculs d'attention des messages précédents.
- ☐ Lorsqu'un nouveau message arrive, le modèle calcule uniquement l'attention du nouveau texte et la combine avec le "cache" existant, ce qui accélère drastiquement le temps de réponse.

Gestion du context et de la mémoire conversationnelle (4/4)

4; La mémoire à long terme : RAG et bases de données (Module 4)

- ☐ Pour dépasser définitivement les limites de la mémoire à court terme, les architectures modernes utilisent le **RAG (Retrieval-Augmented Generation)** ou des bases de données de vecteurs :
- ☐ La conversation est enregistrée dans une base de données externe.
- ☐ Lorsque vous posez une question faisant référence à un vieux souvenir, le système cherche les messages sémantiquement proches dans la base de données.
- ☐ Il extrait ces fragments du passé et les réinjecte comme contexte immédiat dans la requête actuelle.

Évaluation des Performances des Modèles

Mesurer l'intelligence ou la précision d'un LLM nécessite des outils automatisés standardisés (Benchmarks).

Métriques traditionnelles (Automatiques)

- ❑ **BLEU / ROUGE** : Mesurent le taux de superposition de mots entre la réponse de l'IA et une réponse humaine de référence (utilisé pour la traduction et le résumé).
- ❑ **Perplexité** : Calcule la certitude du modèle à prédire le mot suivant. Plus elle est basse, plus le modèle est performant.

Benchmarks de Capacités (Évaluation moderne)

- ❑ **MMLU (Massive Multitask Language Understanding)** : Évalue les connaissances générales à travers des questions à choix multiples (sciences, droit, histoire).
- ❑ **GSM8k / HumanEval** : Évaluent respectivement les capacités de raisonnement mathématique et la génération de code Python fonctionnel.
- ❑ **LMSYS Chatbot Arena** : Classement mondial basé sur le vote d'utilisateurs humains comparant à l'aveugle les réponses de deux modèles (système de score Elo comme aux échecs).

Modèles Propriétaires (Fermés) vs Modèles Ouverts (1/2)

Le marché des LLMs est structurellement divisé en deux grandes philosophies de distribution.

Modèles Propriétaires (Fermés)

Exemples : OpenAI (GPT-4o), Anthropic (Claude 3.5 Sonnet), Google (Gemini 1.5 Pro).

- ❑ **Accès** : Uniquement via une interface web ou une API payante. Les poids du modèle et le code d'entraînement sont secrets.
- ❑ **Avantages** : Performances de pointe, infrastructure gérée par le fournisseur, mises à jour transparentes.
- ❑ **Inconvénients** : Dépendance totale au fournisseur, risques liés à la confidentialité des données, coûts récurrents à l'usage.

Modèles Propriétaires (Fermés) vs Modèles Ouverts (2/2)

Le marché des LLMs est structurellement divisé en deux grandes philosophies de distribution.

Modèles Ouverts (Open Weights / Open Source)

Exemples : Meta (Llama 3), Mistral AI (Mixtral), DeepSeek.

- ❑ **Accès** : Les poids du modèle (les fichiers binaires du réseau de neurones) sont téléchargeables librement.
- ❑ **Avantages** : Confidentialité totale (exécution en local ou sur serveurs privés), personnalisation absolue (Fine-Tuning libre), aucun coût d'API.
- ❑ **Inconvénients** : Nécessite une infrastructure matérielle coûteuse (GPU performants) pour l'hébergement et la maintenance technique.